

Sızma Testi ve Güvenlik Değerlendirme Raporu

Proje Adı: Elektrikli Araç Şarj İstasyonu (EVCS) Simülasyonu

Testi Yapan: Feyza Sağman

Tarih: 25 Aralık 2025

Kapsam: Backend API Servisleri (localhost:8000)

1. Yönetici Özeti (Executive Summary)

Bu çalışmada, Elektrikli Araç Şarj Yönetim Sistemi'nin (EVCS) güvenlik duruşunu test etmek amacıyla kontrollü bir sızma testi gerçekleştirilmiştir. Testler sonucunda, sistemin erişilebilirliğini tamamen durduran Kritik seviyede bir **Dağıtık Hizmet Engelleme (DDoS) Simülasyonu** gerçekleştirilmiş ve sistemin **Uygulama Katmanı DoS (Application Layer DoS)** saldırılarına karşı savunmasız olduğu tespit edilmiştir. Test kapsamında, çoklu iş parçacığı (multi-threading) kullanılarak dağıtık bir botnet ağı simüle edilmiş ve sistem erişilemez hale getirilmiştir. Bu zafiyet, kötü niyetli saldırganların şarj istasyonlarını uzaktan devre dışı bırakmasına olanak tanımaktadır.

Test Ortamı Bilgisi:

- İşletim Sistemi:* Windows 11
- Kullanılan Araçlar:* Python 3.x, Requests Kütüphanesi, VS Code.

2. Tespit Edilen Zafiyet Detayları

Başlık	Detay
Zafiyet Adı	DDoS Zafiyeti (Resource Exhaustion / Kaynak Tüketimi)
Önem Derecesi	● KRİTİK (Critical) CVSS v3.1 Puanı: 7.5 (High) (Bu zafiyet uzaktan sömürülebilir ve erişilebilirliği tamamen bitirir.)

Başlık	Detay
Etkilenen Bileşen	/vulnerable/ddos-target Endpoint'i
HTTP Metodu	POST
Zafiyet Kaynağı	Girdi Denetimi (Input Validation) ve Hız Sınırlaması (Rate Limiting) eksikliği

2.1. Teknik Açıklama

Sistemin /vulnerable/ddos-target adresi, istemciden gelen `requests_count` parametresini herhangi bir üst sınır kontrolü yapmadan işlemektedir. Yazılım, bu parametreye girilen sayı kadar döngüsel işlem yapmakta ve sunucu kaynaklarını (CPU/RAM) tüketmektedir.

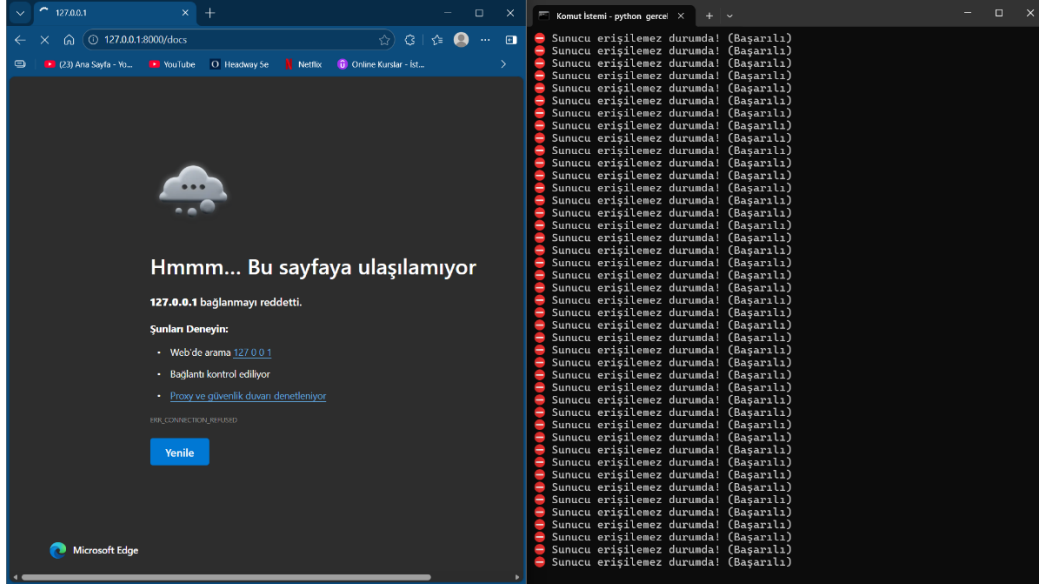
Normal kullanımda 1 olması gereken bu değer, saldırgan tarafından 1000 gibi yüksek değerlere çekildiğinde ve çoklu iş parçacığı (multi-threading) ile tekrarlandığında, sunucu gelen isteklere cevap veremez hale gelmektedir (Timeout/Connection Refused).

3. Sömürü Senaryosu (Exploitation Steps)

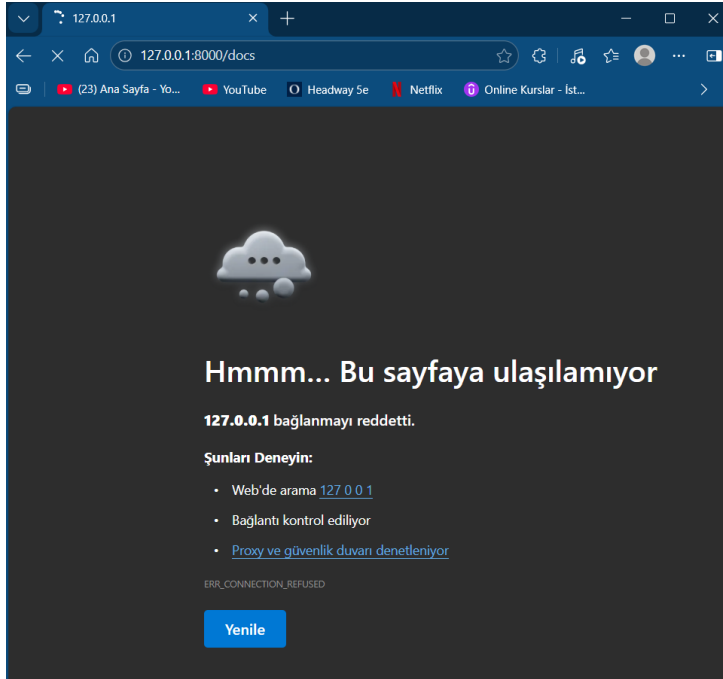
Zafiyetin doğrulanması için aşağıdaki adımlar izlenmiştir (Proof of Concept - PoC):

- Keşif:** Swagger UI üzerinden sistemin API uç noktaları incelendi ve /vulnerable/ddos-target adresinin kimlik doğrulama gerektirmediği görüldü.
- Saldırı Hazırlığı:** Python dili ve `requests` kütüphanesi kullanılarak özel bir saldırı scripti (`gercek_saldiri.py`) geliştirildi.
- Saldırı Anı:** Script, 500 adet eş zamanlı bot (thread) oluşturacak şekilde yapılandırıldı. Her bot, sunucuya `requests_count=1000` parametresiyle ağır iş yükü gönderdi.
- Sonuç:** Saldırı başladıktan saniyeler sonra sunucu yanıt vermeyi kesti.
 - API yanıt süreleri zaman aşımına (Timeout) uğradı.
 - Web arayüzüne erişim tamamen kesildi (`ERR_CONNECTION_REFUSED`).

4. Kanıtlar (Evidence)



- **Şekil 1:** Saldırı scriptinin çalıştırılması ve sunucunun kilitlendiğini gösteren konsol çıktıları (Timeout hataları).



- **Şekil 2:** Tarayıcıdan erişilmeye çalışıldığında alınan "Bu sayfaya ulaşamıyor" hatası.

5. Etki Analizi (Impact Analysis)

Bu zafiyetin gerçek bir senaryoda kullanılması durumunda:

- **Hizmet Kesintisi:** Şarj istasyonları merkezi sunucu ile iletişim kuramaz.
- **Finansal Kayıp:** Müşteriler şarj işlemini başlatamaz veya ödeme yapamaz.
- **İtibar Kaybı:** Şarj ağı operatörü güvenilirliğini yitirir.

6. Çözüm Önerileri (Remediation)

Bu zafiyetin kapatılması için aşağıdaki önlemlerin kod tarafında uygulanması gerekmektedir:

1. Girdi Doğrulama (Input Validation):

requests_count parametresi için maksimum bir sınır (örneğin Max: 5) belirlenmelidir.

```
if requests_count > 5:  
    raise HTTPException(status_code=400, detail="Maksimum işlem sınırı aşıldı.")
```

Python

2. Hız Sınırlaması (Rate Limiting):

Aynı IP adresinden gelen istekler sınırlandırılmalıdır (Örn: Dakikada en fazla 10 istek).

3. Zaman Aşımı (Timeout) Ayarı:

Sunucu, belirli bir süreyi aşan işlemleri otomatik olarak iptal etmelidir.